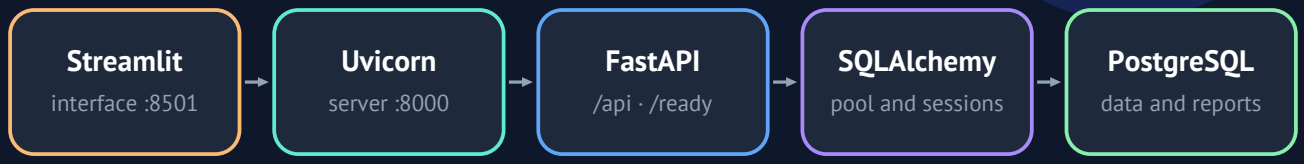
LEARNING PROJECT NO. 2 · FICTIONAL COMPANY INDUSTRIALFLOW

AI Data Analyst

Stages 01–08 · from data to an interface

A visual guide: the uv environment, a FastAPI API, PostgreSQL in Docker, Alembic migrations, SQL, reproducible data, analytics, anomalies, a REST API, an AI agent with tools, Streamlit, charts, tests and CI.



Code computes the numbers; the database itself guards the data.

8 / 10	484 / 484	9 / 9	6	79
course stages done	tests passed	local checks PASS	data tables	packages in uv.lock

Contents 01: 2–11 · 02: 12–20 · 03: 21–23 · 04: 24–26 · 05: 27–29 · 06: 30–32 · 07: 33–36 · 08: 37–39

1 Project in a minute
2 Request path
3 Environment and uv
4 Creating the project
5 Imports and sys.path
6 HTTP and JSON
7 API code walkthrough
8 Settings
9 API tests
10 Ruff and mypy
11 Git and GitHub
12 PostgreSQL in Docker
13 Tables and relations
14 Integrity rules
15 Money and time
16 SQLAlchemy and sessions
17 Alembic migrations
18 /health and /ready
19 Hand-written SQL
20 Database tests
21 Reproducible data
22 Scenarios A01–A05
23 Loading and quality
24 Exact metrics
25 Months and comparisons
26 Group contributions
27 Statistical candidates
28 Isolation Forest
29 Offline scenario evaluation
30 REST API and parameters
31 Sessions and errors
32 Saved reports
33 Agent tools
34 Tool-calling loop
35 Report and evidence
36 Providers and history
37 Streamlit interface
38 Metrics and charts
39 Questions and reports in the UI
40 CI on GitHub Actions
41 Command cheat sheet
42 Common errors
43 Check yourself
44 Verification results

Verified 2026-10-21 · commit 2e34253 + worktree · uv 0.12.23 · Python 3.12.15 · PostgreSQL 17 · macOS, Apple Silicon

Stage guides: docs/STAGE_01.md–STAGE_08.md; instructions: docs/instructions/

1 Project in a minute

What already works, what does not exist yet and where the project is in the course plan.

AI Data Analyst is an AI analyst for **IndustrialFlow**, a fictional company selling pumps, compressors, valves and sensors. A manager asks in plain language; the agent calls analytics tools, computes exact metrics and writes a report. Stages 01–08 are done: an API foundation with checks and CI, a PostgreSQL database, reproducible data, quality checks, exact analytics, anomaly detection, a REST API and an AI agent with tools, evidence checks and run history; a Streamlit interface with metrics, charts and saved reports.

API

FastAPI: /health and /ready; metrics, group contributions and anomalies under /api/...; reports and agent runs. /docs is generated.

Database

PostgreSQL 17 in Docker Compose: 6 data tables, 28 integrity rules, NUMERIC and timestamptz; reports and agent_runs are separate.

Migrations

The schema is created and rolled back only by Alembic; alembic check catches a model without a migration.

Tests

484 pytest tests: 369 unit and 115 on a real PostgreSQL. All pass.

Quality

Ruff checks style and formats code and Markdown examples; mypy checks types in strict mode.

CI

GitHub Actions repeats everything on a clean Linux machine with its own PostgreSQL on every push and pull request.

Where we are in the course plan

1 done **Project setup**	2 done **PostgreSQL and schema**	3 done **Data generation**	4 done **Analytics core**	5 done **Anomaly detection**
6 done **REST API**	7 done **AI agent**	8 done **UI and reports**	9 next **Tests and security**	10 **Docker and release**

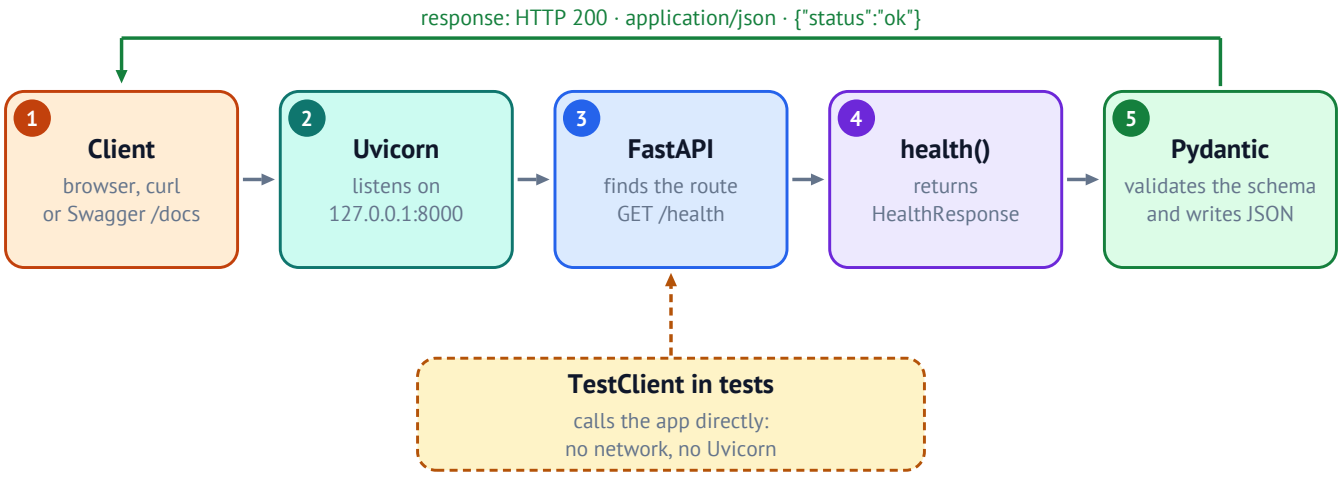
Numbers come from code, not from the LLM. Revenue, average order value, comparisons and anomalies are computed by deterministic SQL or Python. The model understands the question, picks tools and explains the result. That is why database rules, exact money types and tests matter.

Verification limits: the agent was tested with mock and a fake SDK; a live OpenAI call is still untested. The Streamlit UI uses the same API; authentication (09) and release (10) come later.

Code excerpts are shown exactly as they are in the repository, so comments may be in Russian or English. The small sample uses Russian names; generated demo data uses English labels. Detailed stage guides are in Russian (docs/STAGE_*.md); short task instructions are in English (docs/instructions/).

2 Request path

What happens between pressing Execute in /docs and the {"status": "ok"} reply.



Step	Who	What it does
1	Client	Builds an HTTP request GET /health and sends it to 127.0.0.1:8000.
2	Uvicorn	Accepts the connection, parses HTTP and passes the request to the app over ASGI.
3	FastAPI	Finds the handler by method and path. POST gets 405, an unknown path gets 404.
4	health()	Creates a HealthResponse(status="ok") object.
5	Pydantic	Validates the object against the response schema and writes JSON.

Actual server response

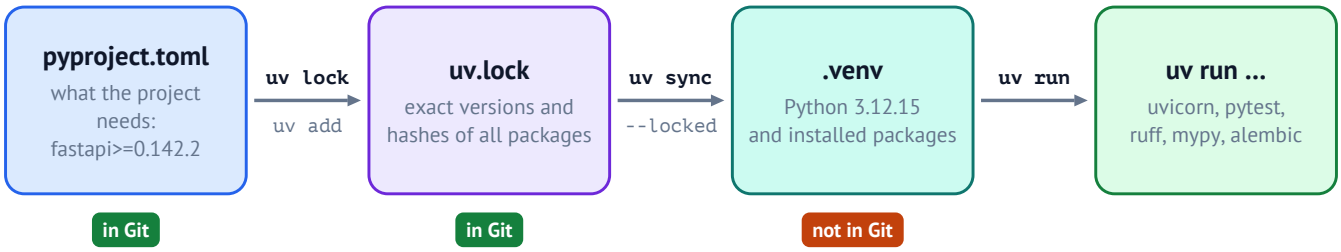
```
$ curl -i http://127.0.0.1:8000/health
HTTP/1.1 200 OK
server: uvicorn
content-length: 15
content-type: application/json

{"status":"ok"}
```

TestClient in tests connects straight to FastAPI (step 3): no network, no Uvicorn and no free port. That is why unit tests run in a fraction of a second.

3 Environment and uv

How uv turns a list of requirements into the same environment on any computer.



A virtual environment `.venv` is a separate folder with its own Python and libraries for this project only, so projects do not interfere. The system `python3` on this Mac is 3.9.6; we leave it alone and run everything with `uv run`.

	pyproject.toml	uv.lock
Written by	A person and uv add	uv only
Contents	Requirements: fastapi>=0.142.2	Exact versions and file hashes
Transitive packages	No	Yes, all 79: starlette, pydantic, anyio…
In Git	Yes	Yes

Project dependencies (versions from uv.lock)

Package	Version	Group	Purpose
`alembic`	`1.20.0`	runtime	schema migrations
`fastapi`	`0.142.2`	runtime	routes, validation, /docs
`openai`	`3.24.0`	runtime	model tools and reports
`plotly`	`7.1.0`	runtime	interactive charts
`psycopg`	`3.3.6`	runtime	PostgreSQL driver
`pydantic-settings`	`2.15.0`	runtime	settings from env and .env
`scikit-learn`	`1.9.1`	runtime	Isolation Forest detector
`sqlalchemy`	`2.1.3`	runtime	models, pool, sessions
`streamlit`	`1.65.0`	runtime	browser interface
`uvicorn`	`0.54.0`	runtime	ASGI server, listens on a port
`httpx2`	`2.13.1`	dev	HTTP client for TestClient
`mypy`	`2.4.0`	dev	type checking
`pytest`	`9.1.1`	dev	running tests
`ruff`	`0.16.10`	dev	linter and formatter
`starlette`	`1.7.0`	fastapi	web foundation of FastAPI
`pydantic`	`2.13.5`	fastapi	models and validation

Package age policy. `[tool.uv] exclude-newer = "2026-10-06T00:00:00Z"` — uv ignores PyPI releases newer than this date. A fresh release may be broken or even malicious; within two weeks such problems are usually found.

4 Creating the project

Which commands created the project, how they differ from the starter task and what each folder holds.

```
# uv 0.12.23 — the same version as pinned in CI
curl -LsSf https://astral.sh/uv/0.12.23/install.sh \
  | env UV_NO_MODIFY_PATH=1 sh

uv init --app --no-package --python 3.12 --author-from none
rm main.py && mkdir -p app tests
uv add fastapi 'uvicorn[standard]' pydantic-settings
uv add --dev pytest ruff mypy httpx2
```

uv init in uv 0.12

Without flags it creates a library package: src/ai_data_analyst/, [build-system] and [project.scripts]. A web service with app/ at the root needs --app --no-package.

httpx2 instead of httpx

Starlette 1.7 warns: “Using httpx with starlette.testclient is deprecated; install httpx2 instead”. TestClient runs on httpx2.

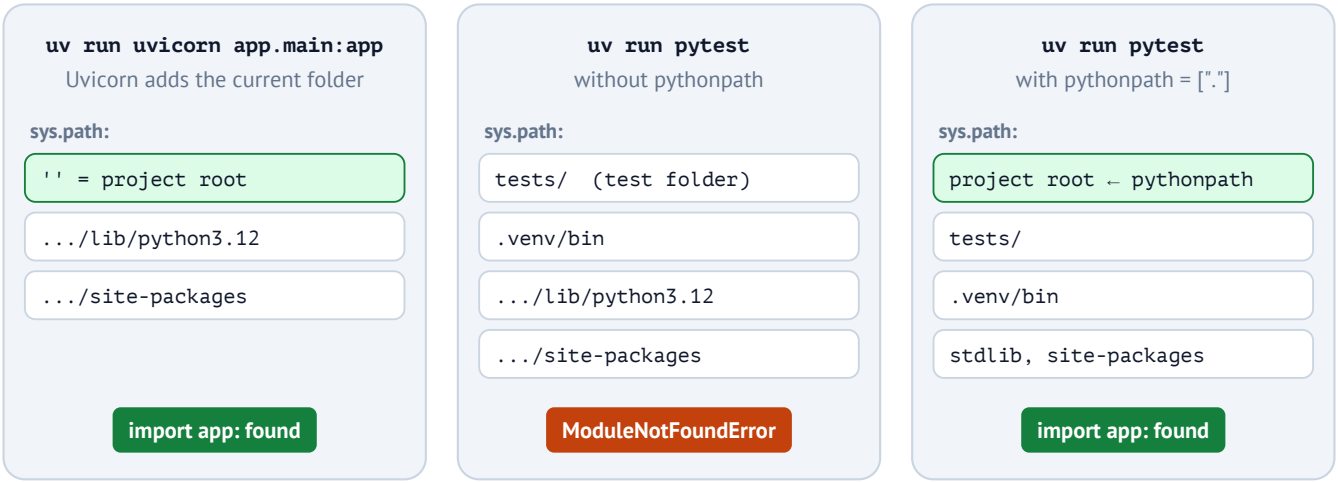
Stage 01 files

File or folder	Purpose
`app/__init__.py`	Makes app a package; holds __version__ = "0.1.0".
`app/config.py`	The Settings class and get_settings().
`app/main.py`	The FastAPI object and service routes.
`tests/`	Tests: API, settings, version; later the database.
`pyproject.toml`	Project metadata, dependencies, pytest/Ruff/mypy/uv settings.
`uv.lock`	Exact versions of all 79 packages.
`.python-version`	3.12 — the Python uv selects.
`.env.example`	Local settings template; .env itself is not in Git.
`.gitignore`	What Git ignores: .venv, caches, .env.
`.github/workflows/ci.yml`	Checks in GitHub Actions.
`docs/`	Guides, instructions, CI/CD, approach, ADRs, log, report, PDFs.
`scripts/`	Builds this PDF in Russian and English.
`AGENTS.md`	Working rules for the project.

5 Imports and sys.path

Why the server finds the app package and pytest without a setting does not.

A module is one `.py` file. **A package** is a folder of modules; `__init__.py` runs on the first import. **sys.path** is the list of folders where Python looks for modules. `from app.main import app` works only if one of these folders contains the app folder.



Actual error before the setting

```
$ uv run pytest
ImportError while importing test module '.../tests/test_health.py'.
E   ModuleNotFoundError: No module named 'app'
Interrupted: 3 errors during collection
```

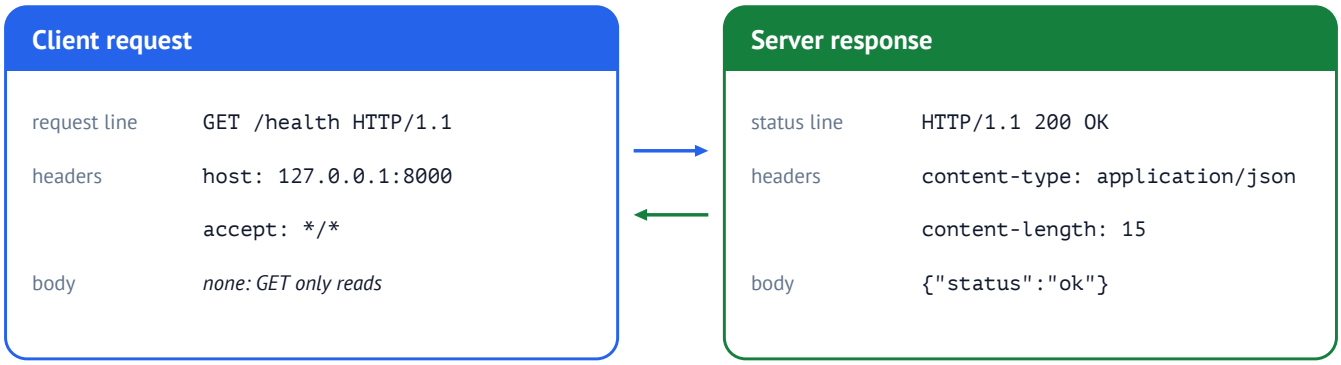
The fix in pyproject.toml

```
[tool.pytest.ini_options]
testpaths = ["tests"]
pythonpath = ["."]
```

Why. The project was created with --no-package: .venv gets only the dependencies, app itself is not installed. Alternatives are python -m pytest (-m adds the current folder) or packaging the project. An explicit setting works however pytest is started.

6 HTTP and JSON

What a request and a response consist of and which codes the project uses.



Method	Meaning	In this project
`GET`	Read data without changing it	GET /health, GET /ready; later GET /api/metrics/revenue
`POST`	Create an object or run an action	later: a question to the AI agent
`PUT / PATCH`	Replace or partially update	not used yet
`DELETE`	Delete	not used yet

Code	Meaning	When you see it (verified)
200 OK	Success	GET /health → 200 OK
404 Not Found	No such path	GET /nope → 404 Not Found
405 Method Not Allowed	Path exists, wrong method	POST /health → 405 Method Not Allowed, allow: GET
422 Unprocessable Content	Invalid input	arrives in Stage 06 with query parameters
500 Internal Server Error	Server error	experiment in section 7
503 Service Unavailable	Service not ready	GET /ready without a database — section 18

JSON is a text data format: objects {...}, arrays [...], strings, numbers, true / false / null. FastAPI turns a Pydantic model into JSON itself, and `response.json()` in a test turns JSON back into a Python dict.

7 API code walkthrough

The /health route line by line and an experiment with a wrong reply.

```
app = FastAPI(
    title=get_settings().app_name,
    description="AI-powered business analytics platform for IndustrialFlow",
    version=__version__,
    lifespan=lifespan,
)


class HealthResponse(BaseModel):
    """Ответ проверки живости: процесс запущен и отвечает на HTTP-запросы."""

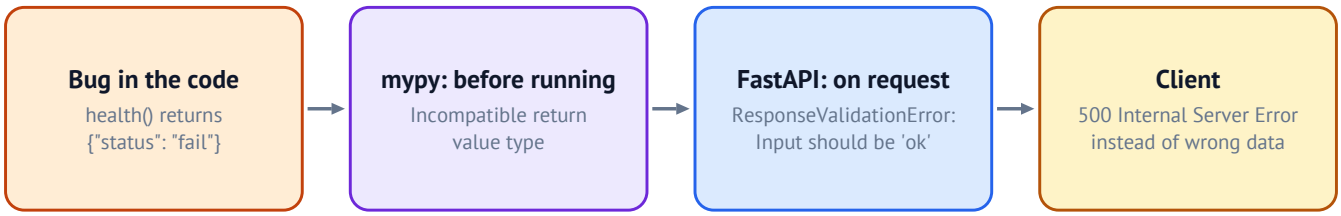
    # Literal допускает ровно одно значение: ответ с другим FastAPI не отправит.
    status: Literal["ok"]


@app.get("/health")
def health() -> HealthResponse:
    """Сообщить, что API работает; базу данных этот маршрут не проверяет."""
    return HealthResponse(status="ok")
```

Fragment	What it does
`app = FastAPI(...)`	The application object. Uvicorn finds it (app.main:app) and TestClient calls it. title and version go to /docs; lifespan closes the database pool on shutdown.
`class HealthResponse`	A Pydantic model describing the reply. Literal["ok"] allows exactly one value.
`@app.get("/health")`	Registers the function for GET + /health. Other methods get 405.
`-> HealthResponse`	The return annotation becomes the response schema: it drives /docs and validates the function's result.
`def, not async def`	FastAPI runs a plain function in a worker thread. We return to async in Stage 06.

Experiment: a wrong reply

In a temporary copy of the project health() returned `{"status": "fail"}`. Actual results:



Two layers of protection: types catch the bug in code, Pydantic catches it in data. A wrong reply never reaches the client silently. For analytics this matters most: an explicit error beats a wrong number in a report.

Model → JSON → dict

FastAPI turns a model object into JSON text, and a test turns JSON back into a Python dict. The values come from the real application while the PDF is built:

```
>>> HealthResponse(status="ok").model_dump_json()   # model → JSON
'{"status":"ok"}'
>>> client.get("/health").json()                    # JSON → dict
{'status': 'ok'}
```

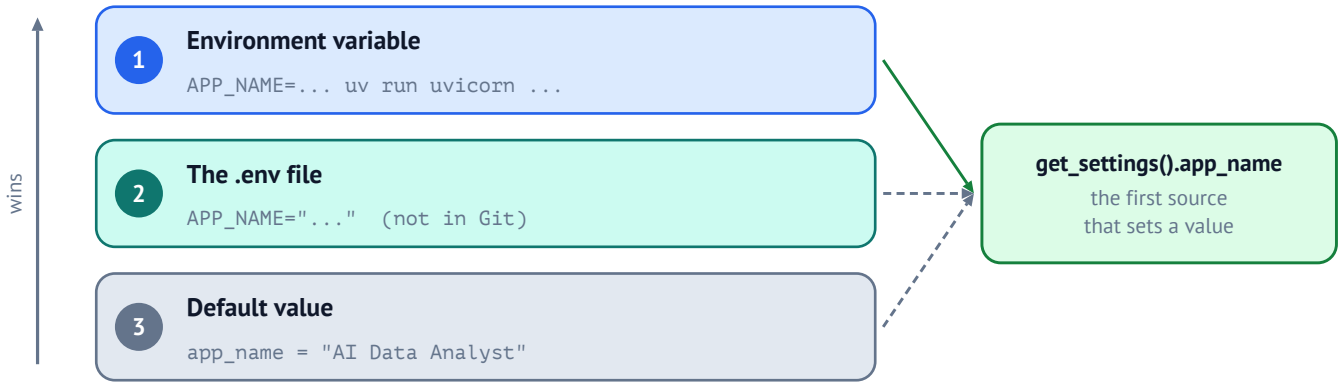
OpenAPI and the /docs page

FastAPI collects routes and models into an OpenAPI document (`GET /openapi.json`). The /docs page is Swagger UI drawing that document. The HealthResponse schema from the app; the model docstring became the description field:

```
{
  "properties": {
    "status": {
      "type": "string",
      "const": "ok",
      "title": "Status"
    }
  },
  "type": "object",
  "required": [
    "status"
  ],
  "title": "HealthResponse",
  "description": "Ответ проверки живости: процесс запущен и отвечает на HTTP-запросы."
}
```

8 Settings

Configuration lives in the environment, not in the code.




```
class Settings(BaseSettings):
    """Конфигурация, которая читается при создании объекта настроек.

    Приоритет источников: переменная окружения → строка в .env → значение
    по умолчанию ниже. Регистр имён не важен: поле app_name читается из
    переменной APP_NAME.
    """

    # extra="ignore": в .env лежат и ключи Docker (POSTGRES_*), которые
    # к этим настройкам не относятся; без него Pydantic отклонил бы их.
    # hide_input_in_errors: адрес БД содержит пароль — его нельзя выводить
    # в тексте ошибки проверки.
    model_config = SettingsConfigDict(
        env_file=".env",
        env_file_encoding="utf-8",
        extra="ignore",
        hide_input_in_errors=True,
    )

    # Название сервиса в OpenAPI-схеме и на странице /docs.
    app_name: str = "AI Data Analyst"
    ...


@lru_cache
def get_settings() -> Settings:
    """Прочитать настройки один раз на процесс и дальше отдавать тот же объект.

    Функция вместо глобального объекта: настройки читаются при первом
    обращении, а тесты могут сбросить кеш через get_settings.cache_clear().
    """
    return Settings()
```

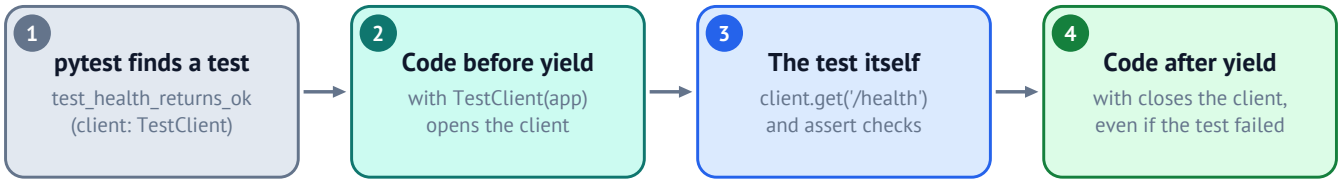
Test	What it pins down
`test_default_app_name`	Without env and .env the code default wins.
`test_env_file_overrides_default`	A line in .env replaces the default.
`test_environment_variable_beats_env_file`	An environment variable beats .env.

extra="ignore" — .env also holds Docker variables (POSTGRES_*) unrelated to Settings. **get_settings()** reads settings once per process, so .env changes apply after a server restart. Stage 02 added database_url — section 12.

9 API tests

How pytest finds tests, what a fixture is and what each Stage 01 test proves.

Pytest finds `test_*.py` files in tests/ and runs functions named `test_...`. Inside is a plain assert: when it fails, pytest shows which values differ. A fixture prepares a test; the test gets it by declaring a parameter with the same name.




```
@pytest.fixture
def client() -> Iterator[TestClient]:
    """Открыть тестовый клиент: запросы идут прямо в приложение, без сети.

    Фикстура выполняется перед каждым тестом, который принимает параметр
    client, а код после yield закрывает клиент после теста.
    """
    with TestClient(app) as test_client:
        yield test_client
```

Test	What it proves
`test_health_returns_ok`	GET /health → 200 and exactly {"status": "ok"}
`test_health_rejects_post`	POST /health → 405
`test_openapi_describes_health`	The schema knows the version, path and status
`test_default_app_name`	Code default
`test_env_file_overrides_default`	.env beats the default
`test_environment_variable_beats_env_file`	Environment beats .env
`test_version_matches_pyproject`	app.__version__ matches pyproject.toml

These are the 7 Stage 01 tests. The project now has 369 unit and 115 integration tests. Checks for Stages 02–05 are in sections 20, 23, 26 and 29.

10 Ruff and mypy

Two tools that find bugs before the program runs.

Ruff rule	What it finds
E	PEP 8 style violations
F	Errors: unused imports, undefined names
I	Import order
UP	Old syntax that is simpler in Python 3.12
B	Common traps such as a mutable default argument

`ruff check .` finds problems, `--fix` fixes the safe ones; `ruff format --check .` checks formatting, including Python blocks in Markdown. **mypy strict** requires annotations on every function; it now also checks `migrations/`.

Example: what Ruff finds

```
from app.config import settings
import os
from typing import List


def names(items: List[str], seen=[]) -> List[str]:
    return [item for item in items if item not in seen]
```

```
$ uv run ruff check --output-format concise example.py
example.py:1:1: I001 [*] Import block is un-sorted or un-formatted
example.py:1:24: F401 [*] `app.config.settings` imported but unused
example.py:2:8: F401 [*] `os` imported but unused
example.py:3:1: UP035 `typing.List` is deprecated, use `list` instead
example.py:6:18: UP006 [*] Use `list` instead of `List` for type annotation
example.py:6:34: B006 Do not use mutable data structures for argument defaults
example.py:6:41: UP006 [*] Use `list` instead of `List` for type annotation
Found 7 errors.
[*] 5 fixable with the `--fix` option
    (1 hidden fix can be enabled with the `--unsafe-fixes` option).
```

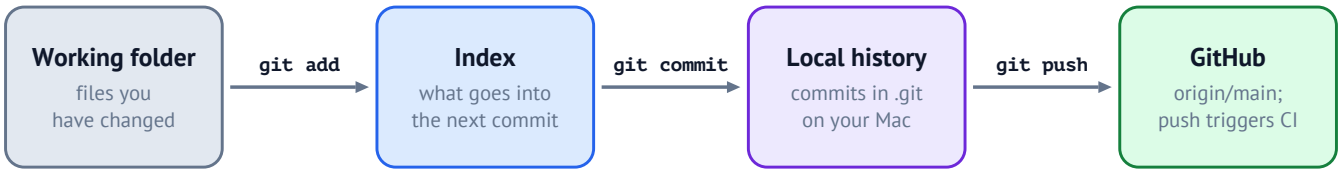
I001 import order, F401 unused import, UP035 and UP006 List instead of list, B006 a shared default list that accumulates values between calls. [*] means Ruff fixes it with --fix.

Example: what mypy finds

```
$ uv run mypy app
app/main.py:30: error: Incompatible return value type
    (got "dict[str, str]", expected "HealthResponse")  [return-value]
Found 1 error in 1 file (checked 3 source files)
```

11 Git and GitHub

What the repository stores and how changes reach GitHub.



In Git

Code in app/, tests/, migrations/, sql/, pyproject.toml, uv.lock, compose.yaml, alembic.ini, .env.example, the workflow, docs and PDFs.

Not in Git

.venv (rebuilt by uv sync), tool caches, .env with the database password and future keys, PostgreSQL data in the Docker volume.

Publishing

Repository: **merkertg/ai-data-analyst**, branch main. For a new project: **New repository** on github.com without README, .gitignore or license, then:

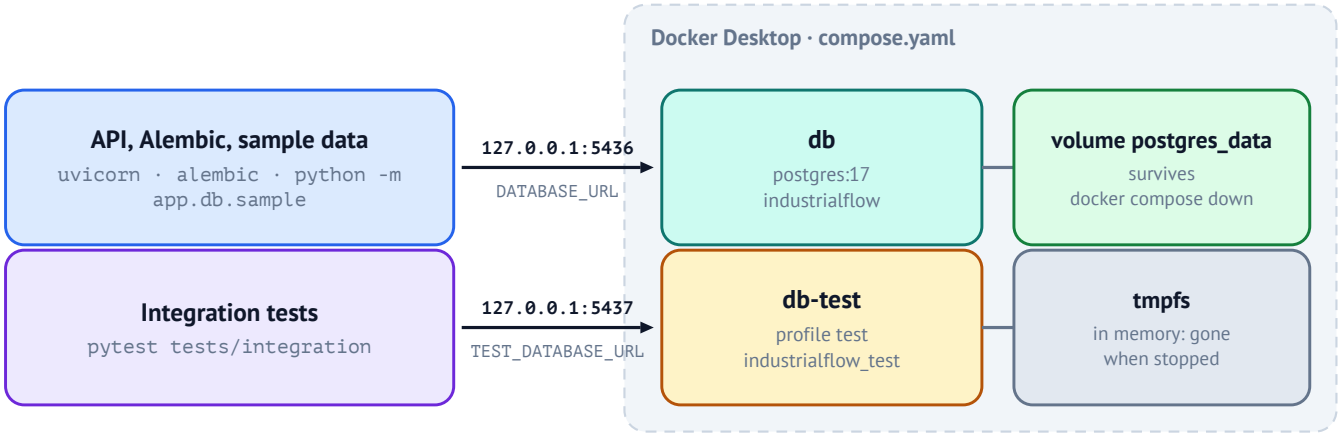
```
git remote add origin git@github.com:<login>/<repository>.git
git push -u origin main
```

Then every change: `git add` → `git commit` → `git push origin main`. The **Actions** tab shows the CI result; only it confirms the remote check.

> Commit messages are in English and imperative: “Add …”, “Fix …”. Each commit is a finished step after which all checks pass.

12 PostgreSQL in Docker

Where the database lives, how the app and tests connect to it and what survives a restart.



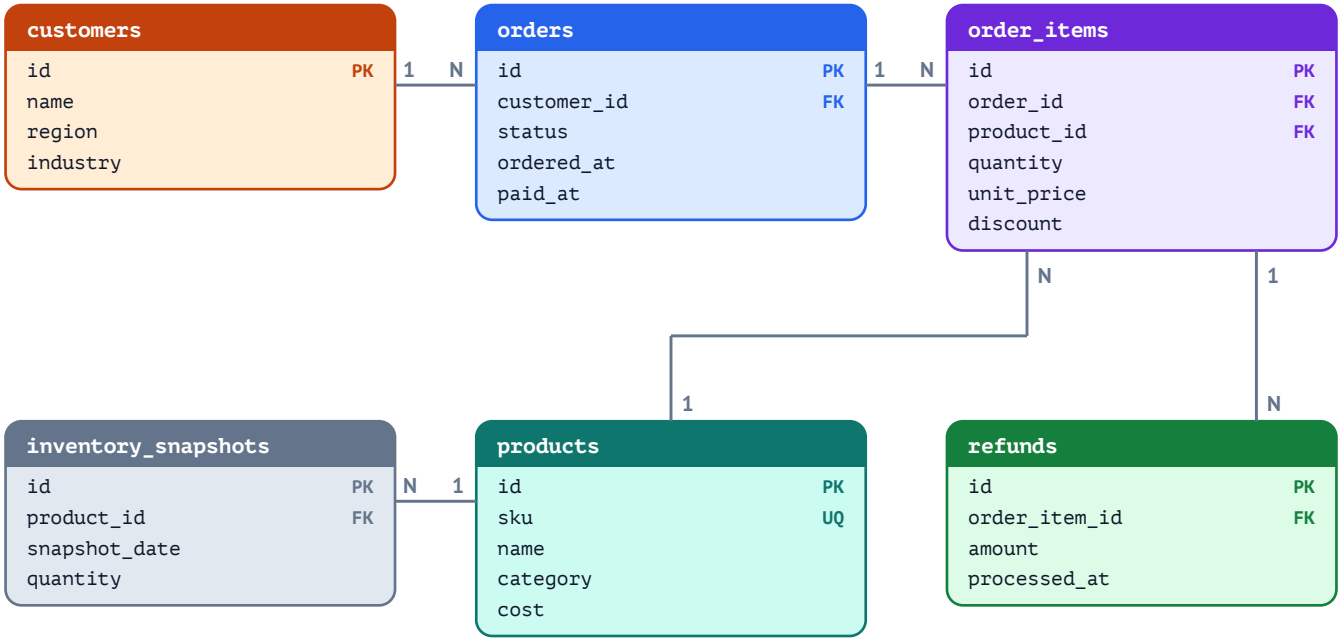

```
cp .env.example .env
docker compose up --detach --wait db
uv run alembic upgrade head
uv run python -m app.db.sample
docker compose exec db psql -U analyst -d industrialflow
docker compose stop
```

Decision	Why
Port bound to 127.0.0.1 only	The database is not reachable from the local network.
Ports 5436 / 5437	No clash with other PostgreSQL on the Mac (the first project uses 5433/5434).
Image pinned by digest	The postgres:17 tag can be overwritten; a digest cannot. CI uses the same digest.
healthcheck · pg_isready	--wait blocks until the database really accepts connections.
Password from .env	Without .env Compose stops with a hint; the password never reaches Git.

Verified: after docker compose down and a new up the container changed, while 5 orders and schema version 0001 stayed in the volume. **docker compose down -v deletes the volume with all data.**

13 Tables and relations

Six tables, primary and foreign keys, one-to-many relations.



PK primary key, FK foreign key, UQ unique value

Table	What it stores
customers	Customers — industrial companies: region and industry.
products	Catalogue: unique SKU, category, unit cost.
orders	Order: customer, status, when placed and when paid.
order_items	Lines: product, quantity, sale price, discount.
refunds	Money refunds per line and when they were processed.
inventory_snapshots	Stock of a product at the end of a day.

Normalisation. One order holds many products, so orders and lines are separate tables: the customer and payment date are stored once in orders, the region once in customers. The sale price sits on the line, not the product: history must keep the price actually charged.

14 Integrity rules

28 named constraints and 7 indexes: PostgreSQL itself rejects an invalid row.

Kind	Count	Examples in the project
CHECK	14	quantity > 0; 0 ⩽ discount < 1; cost ⩾ 0; amount > 0; paid orders have paid_at, others do not; paid_at ⩾ ordered_at
UNIQUE	3	sku; one line per product per order; one snapshot per day
FOREIGN KEY	5	no reference to a missing customer, product or order
PRIMARY KEY	6	id of every table
NOT NULL	—	every column except paid_at

Relation	On delete	Why
order → lines	CASCADE	Lines make no sense without their order.
customer → orders	RESTRICT	A customer with sales history cannot be deleted.
product → lines, stock	RESTRICT	A sold product stays in the catalogue.
line → refunds	RESTRICT	Processed money never vanishes; an order delete cascade stops at a refund.

Actual PostgreSQL errors

```
ERROR:  new row for relation "order_items" violates check constraint
        "ck_order_items_quantity_positive"
DETAIL:  Failing row contains (7, 1, 3, 0, 100.00, 0.0000).

ERROR:  update or delete on table "products" violates foreign key constraint
        "fk_inventory_snapshots_product_id_products" on table "inventory_snapshots"
DETAIL:  Key (id)=(1) is still referenced from table "inventory_snapshots".
```

Constraint names

Convention in app/db/base.py: ck_, fk_, uq_, ix_ + table + columns. The name in an error tells the rule; tests assert exactly that name.

What the database does not check

Cross-table rules: refunds not above the line total, refunds only for paid orders. Stage 03 data quality checks cover them.

15 Money and time

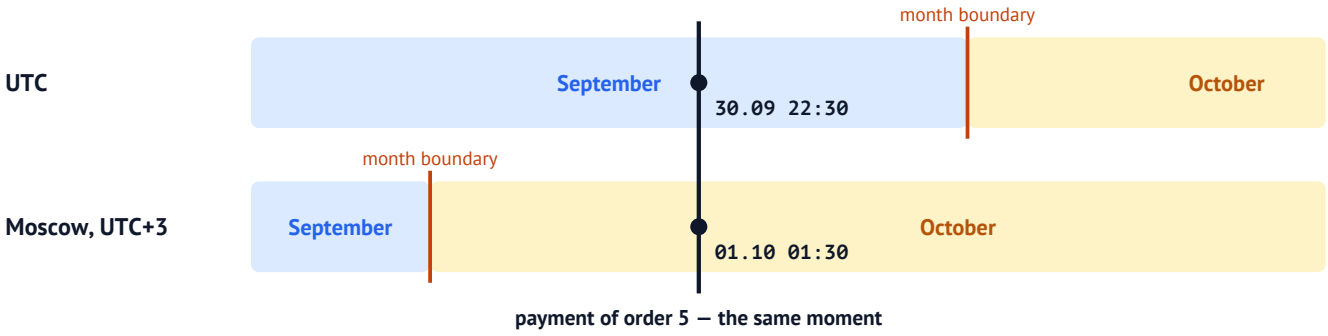
Why money is NUMERIC and Decimal, and time is timestamptz with Moscow months.

```
>>> 0.1 + 0.2
0.30000000000000004
>>> from decimal import Decimal
>>> Decimal("0.1") + Decimal("0.2")
Decimal('0.3')
```

Where	Type	Meaning
Price, cost, refund	NUMERIC(12, 2)	up to 9,999,999,999.99 RUB to the kopeck
Discount	NUMERIC(5, 4)	a fraction: 0.1250 = 12.5%
Python	Decimal	SQLAlchemy reads NUMERIC as Decimal
Line amount	ROUND(q × price × (1 − d), 2)	each line rounded to the kopeck

A too large value is rejected with numeric field overflow, not distorted — the test test_money_overflow_is_rejected checks it.

One moment, two months



timestamptz stores a **moment**, not a wall clock. Order 5 was paid on 1 October at 01:30 Moscow time = 30 September 22:30 UTC. The company's reporting month is Moscow time, so queries use `AT TIME ZONE 'Europe/Moscow'`. In UTC September would be 328,000.00 instead of 310,000.00 — the test test_month_depends_on_time_zone checks it.

16 SQLAlchemy and sessions

Model classes describe tables; the pool and sessions manage connections and transactions.

```
class Order(Base):
    """Заказ клиента. Оплата фиксируется временем paid_at."""

    __tablename__ = "orders"
    __table_args__ = (
        CheckConstraint(
            "status IN ('pending', 'paid', 'cancelled')", name="status_allowed"
        ),
        CheckConstraint(
            "(status = 'paid') = (paid_at IS NOT NULL)", name="paid_at_matches_status"
        ),
        CheckConstraint("paid_at >= ordered_at", name="paid_after_ordered"),
        Index(None, "customer_id"),
        Index(None, "paid_at"),
    )

    id: Mapped[int] = mapped_column(Integer, primary_key=True)
    customer_id: Mapped[int] = mapped_column(
        ForeignKey("customers.id", ondelete="RESTRICT")
    )
    status: Mapped[str] = mapped_column(
        String(16), default=OrderStatus.PENDING, server_default=OrderStatus.PENDING
    )
    ordered_at: Mapped[datetime] = mapped_column(DateTime(timezone=True))
    paid_at: Mapped[datetime | None] = mapped_column(DateTime(timezone=True))

    customer: Mapped[Customer] = relationship(back_populates="orders")
    items: Mapped[list["OrderItem"]] = relationship(
        back_populates="order", cascade="all, delete-orphan", passive_deletes=True
    )
```

Element	Meaning
`Mapped[int]`	Type for Python and mypy; without None it is NOT NULL.
`Mapped[datetime | None]`	The column may be NULL.
`mapped_column(...)`	Database type and column rules.
`__table_args__`	Table-level constraints and indexes.
`relationship(...)`	Object links: order.customer, order.items. Not a column.

Connection pool and sessions

Engine

The connection pool. Created lazily: import and /health never touch the database. pool_pre_ping drops broken connections.

Session

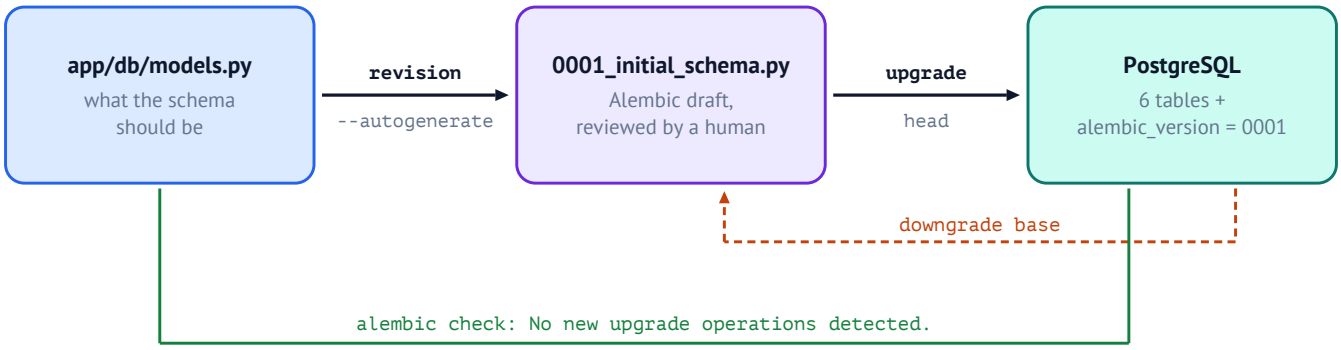
A unit of work: flush() sends rows, commit() makes them permanent, rollback() undoes the whole transaction.

Transaction

All or nothing. There is no hidden commit: code that writes data commits it explicitly.

17 Alembic migrations

The schema changes only through migrations: history, rollback and the same database for everyone.



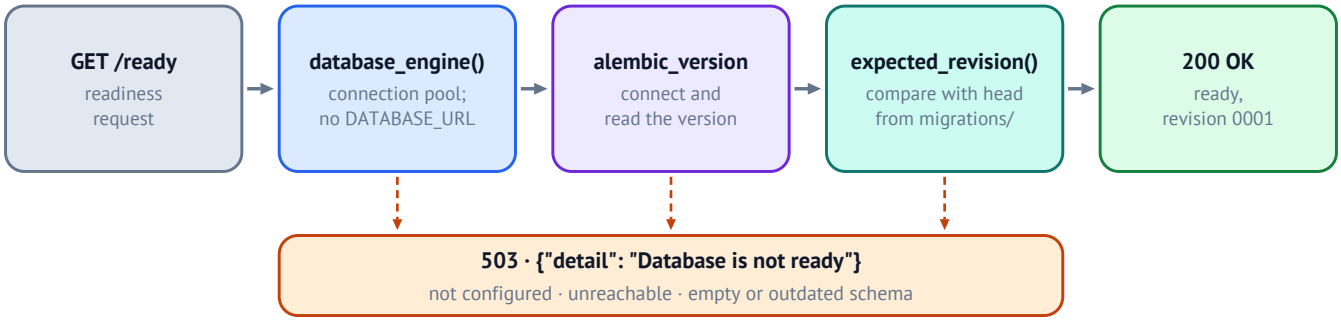

```
$ uv run alembic upgrade head
INFO  [alembic.runtime.migration] Running upgrade  -> 0001, Initial schema: ...
$ uv run alembic current
0001 (head)
$ uv run alembic check
No new upgrade operations detected.
```

Command	What it does
`alembic upgrade head`	Apply all pending migrations
`alembic downgrade -1`	Roll back one migration
`alembic downgrade base`	Roll back everything — data is lost
`alembic current`	Version in the database
`alembic history`	List of migrations
`alembic check`	Are there model changes without a migration
`alembic upgrade head --sql`	Show SQL without running it

Why not create_all(). It only creates missing tables and never changes existing ones. **Autogenerate is a draft**: a human reads the file. **An applied migration is never edited** — every schema change is a new migration 0002, 0003…

18 /health and /ready

Whether the process is alive and whether it can take requests are two different questions.



	/health	/ready
Question	Is the process alive?	Can it take requests?
Touches the database	No	Yes: connection and schema version
Without a database	200	503
Who uses it	restarting a dead process	deciding whether to send traffic

Actual check on a real server

```
# база работает / database running
HTTP/1.1 200 OK                   {"status":"ready","revision":"0001"}
# docker compose stop db
HTTP/1.1 503 Service Unavailable  {"detail":"Database is not ready"}
# docker compose start db
HTTP/1.1 200 OK                   {"status":"ready","revision":"0001"}
```

The third reply shows pool_pre_ping: the pool dropped broken connections and opened new ones without restarting the API. The 503 reply is the same for every cause — the database address and driver errors never leak out.

19 Hand-written SQL

Four queries on the sample dataset; every answer was computed in advance.

Order	Status	Contents	Total
1	paid 5 Sep	2 pumps × 100,000 − 10%, 5 sensors × 2,000	190 000.00
2	paid 20 Sep	1 pump × 120,000	120 000.00
3	cancelled	10 valves × 4,500	—
4	pending	3 sensors × 2,000	—
5	paid 1 Oct 01:30 MSK	4 valves × 4,500	18 000.00

Refunds on the sensors of order 1: 1,000.00 (10 September) and 500.00 (2 October). Running a file: `docker compose exec -T db psql -U analyst -d industrialflow < sql/02_net_revenue_by_month.sql`

01 · Paid order totals: JOIN and GROUP BY

```
 order_id |        customer        |        paid_at         | items | order_total
----------+------------------------+------------------------+-------+-------------
        1 | ООО «ТехМонтаж-Север»  | 2026-09-05 09:00:00+00 |     2 |   190000.00
        2 | АО «Южный химкомбинат» | 2026-09-20 06:30:00+00 |     1 |   120000.00
        5 | ООО «ТехМонтаж-Север»  | 2026-09-30 22:30:00+00 |     1 |    18000.00
```

psql prints times in UTC (+00): 09:00 UTC = 12:00 Moscow time.

02 · Net revenue by month: CTE and FULL OUTER JOIN

```
WITH sales AS (
    SELECT
        date_trunc('month', o.paid_at AT TIME ZONE 'Europe/Moscow')::date AS month,
        SUM(ROUND(oi.quantity * oi.unit_price * (1 - oi.discount), 2)) AS paid_sales
    FROM orders AS o
    JOIN order_items AS oi ON oi.order_id = o.id
    WHERE o.status = 'paid'
    GROUP BY 1
),
refunded AS (
    SELECT
        date_trunc('month', r.processed_at AT TIME ZONE 'Europe/Moscow')::date AS month,
        SUM(r.amount) AS refunds
    FROM refunds AS r
    GROUP BY 1
)
SELECT
    COALESCE(s.month, r.month) AS month,
    COALESCE(s.paid_sales, 0) AS paid_sales,
    COALESCE(r.refunds, 0) AS refunds,
    COALESCE(s.paid_sales, 0) - COALESCE(r.refunds, 0) AS net_revenue
FROM sales AS s
FULL OUTER JOIN refunded AS r ON r.month = s.month
ORDER BY month;
```

```
   month    | paid_sales | refunds | net_revenue
------------+------------+---------+-------------
 2026-09-01 |  310000.00 | 1000.00 |   309000.00
 2026-10-01 |   18000.00 |  500.00 |    17500.00
```

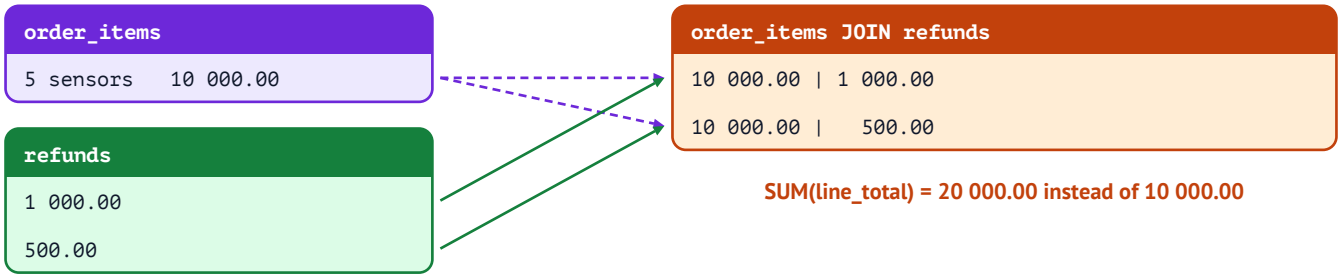
Course rule: Net Revenue = Paid Sales − Processed Refunds; a refund reduces revenue of the month it is processed in.

03 · Sales by category: a window function

```
     category       | paid_sales | share_percent
--------------------+------------+---------------
 Насосы             |  300000.00 |          91.5
 Запорная арматура  |   18000.00 |           5.5
 Датчики            |   10000.00 |           3.0
```

SUM(...) OVER () sees the total of all result rows without collapsing them — that is how a category share is computed.

04 · The double-counting trap




```
 naive_sales | correct_sales | refunds
-------------+---------------+---------
   338000.00 |     328000.00 | 1500.00
```

A one-to-many JOIN repeats the line for each of its refunds: 338,000.00 instead of 328,000.00. Rule: first reduce the “many” table to one row per key (or sum separately, as query 02 does), then join.

20 Database tests

369 unit tests without a database and 115 integration tests on a real PostgreSQL.



Stage 01–02 file	Tests	What it checks
test_config.py	10	settings priority; driver and database name; password hidden
test_ready.py	4	/ready without or with an unreachable database → 503
test_session.py	3	lazy pool, DatabaseNotConfigured
test_models.py	5	6 tables, NUMERIC(12,2), delete rules, names
integration/test_migrations.py	5	upgrade, downgrade, alembic check, /ready
integration/test_constraints.py	23	20 invalid rows, cascade, Decimal, timestamptz
integration/test_queries.py	6	4 SQL files and time zone on the sample data

```
@pytest.mark.parametrize("case", CASES, ids=list(CASES))
def test_invalid_row_is_rejected(db_session: Session, case: str) -> None:
    """Плохая строка вызывает IntegrityError с именем нужного ограничения."""
    build, constraint = CASES[case]
    db_session.add(build(base_rows(db_session)))

    with pytest.raises(IntegrityError) as error:
        db_session.flush()

    assert violated_constraint(error.value) == constraint
```

```
docker compose --profile test up --detach --wait db-test
export TEST_DATABASE_URL=\
  'postgresql+psycopg://analyst:analyst_test@127.0.0.1:5437/industrialflow_test'
uv run pytest
uv run pytest -m integration
```

The address comes only from TEST_DATABASE_URL and only a *_test database: tests reset the schema and must not touch the development database. Without it: integration tests skip locally and fail on CI (CI=true).

21 Reproducible data

The same seed and parameters produce identical files without a database.

The IndustrialFlow demo has **20,000 orders, 640 customers, 64 products and 8 regions** for January–September 2026. The company and transactions are fictional. Reporting months use Europe/Moscow.

```
uv run python -m app.data.generate \
  --output data/demo --seed 42 --orders 20000
```

The output directory must be new or empty. The generator refuses to overwrite a bundle. Use another path for a repeat run, such as data/demo-repeat.

What the output directory contains

File	Contents
`customers.csv, products.csv`	Customers and the product catalogue.
`orders.csv, order_items.csv`	Orders and their lines.
`refunds.csv`	Processed monetary refunds.
`inventory_snapshots.csv`	Daily product stock snapshots.
`manifest.json`	Schema version, seed, row counts and CSV integrity hashes.
`ground_truth.json`	Baseline and observed scenario evidence, for evaluation only.

Why the result repeats

A local random.Random(seed) defines the sampling sequence. Decimal computes money, and CSV stores decimal text. NumPy and Faker are unnecessary. The same parameters and generator version give equal bytes; a different seed changes individual orders.

```
uv run python -m app.data.generate \
  --output data/demo-repeat --seed 42 --orders 20000
diff -r data/demo data/demo-repeat
```

No diff output and exit code 0 confirm equal files. Generated bundles are excluded from Git.

22 Scenarios A01–A05

An unusual business event can fully satisfy the data integrity rules.

The generator first creates baseline data, then injects five scenarios. These are **valid transactions** that must pass quality checks. Later analytics must find their effects in the database tables.

ID	When and where	Change
A01	September, paid pump lines	Quantity = 40% of baseline, rounded down, minimum 1.
A02	August, South, paid valves	A 35% discount.
A03	July, paid compressor lines	An extra refund of 30% of the rounded line amount.
A04	10–19 September, pump ID 1	Inventory snapshot quantity is zero.
A05	One paid sensor line in June	Quantity is 500.

A01 and A04 do not prove causation. Reduced pump quantities and zero stock for one product overlap in time. That motivates a hypothesis to test, not a proven explanation.

What the evaluation answer key means

ground_truth.json stores baseline and observed values computed by code. Tests and later analytics evaluation use it. **The future LLM agent must not read it:** it must obtain numbers from SQL/Python tools over the database.

Separate two questions. “Do refunds exceed the line amount?” checks validity. “Why did refunds rise?” is an analytics question. Passing quality checks does not rule out unusual business events.

23 Loading and quality

One transaction imports the bundle; a separate command only reads it.

```
docker compose up --detach --wait db
uv run alembic upgrade head
uv run python -m app.data.load --input data/demo
uv run python -m app.data.quality
```

All six domain tables must be empty. Choose the small app.db.sample dataset or the full demo before loading. The loader refuses existing rows and provides no reset option.

What protects the import

Bundle and manifest validation detect incomplete or changed files. Customers and products are imported before rows that reference them. One transaction commits the entire bundle; an error rolls it all back. Normal CLI commands use DATABASE_URL from the project settings.

What quality checks

Rule	Violation example
Every order has lines	An order exists without any order lines.
Total refunds ⩽ line amount	A 100.00 line has two 60.00 refunds, totalling 120.00.
Refund only for a paid order	A refund on a pending or cancelled order.
Refund is not before payment	Payment on 12 July, refund on 11 July.

Line amount = ROUND(quantity × unit_price × (1 − discount), 2). quality never changes rows: exit 0 means the rules passed; exit 1 means violations were found. PostgreSQL constraints continue to guard row validity, foreign keys and uniqueness.

Integration tests: TEST_DATABASE_URL only, with a database name ending in _test. Tests never use the development .env database. Actual results are in section 44 and docs/VERIFICATION.md; remote GitHub Actions is confirmed separately.

24 Exact metrics

SQL computes money; Python Decimal preserves exact results.

Stage 04 adds **app/analytics/** and a JSON CLI without new dependencies. PostgreSQL and Decimal compute the metrics. No LLM is involved; ADR 0004 defers Pandas and DuckDB.

Field	Rule
`paid_sales`	Paid-order sales recognised by paid_at.
`refunds`	Refunds by processed_at, only on paid orders.
`net_revenue`	`paid_sales − refunds`
`paid_orders`	Each paid order once.
`average_order_value`	Sales / orders, before refunds; null without paid orders.

Periods and rounding

[start, end) uses Europe/Moscow dates: start included, end excluded. September: 2026-09-01 → 2026-10-01. Each line is rounded to two decimal places **before summing**: ROUND(quantity × unit_price × (1 − discount), 2). Money is NUMERIC/Decimal, in RUB excluding VAT.

> **Sales and refunds use separate aggregates.** Multiple refunds cannot duplicate a sale. Orders are counted from orders, not joined lines. An October refund for a September purchase reduces October revenue.

```
uv run python -m app.analytics \
  --start 2026-09-01 --end 2026-10-01
```

JSON uses decimal strings and null for undefined values. All CLI queries share one snapshot in a REPEATABLE READ READ ONLY transaction. DATABASE_URL comes from settings; no API server is needed.

25 Months and comparisons

Empty months stay in the series; a zero denominator gives no percentage.

monthly_metrics creates calendar buckets, including empty months. Partial first and last months are clipped to the request; the limit is 120 buckets. No paid orders means null AOV. Refund-only periods can have negative net revenue.

Manual example: app.db.sample

Metric	September	October
Paid sales	310000.00	18000.00
Refunds	1000.00	500.00
Net revenue	309000.00	17500.00
Paid orders	2	1
Average order value	155000.00	18000.00

This is the small Stage 02 fixture, **not the 20,000-order demo**. October includes a 500.00 refund for a September sale.

```
uv run python -m app.analytics \
  --start 2026-10-01 --end 2026-11-01 \
  --compare-start 2026-09-01 --compare-end 2026-10-01 \
  --dimension category
```

Absolute and percentage changes

Delta = current − previous. Percentage = delta / abs(previous) × 100, rounded to two places with ROUND_HALF_UP. Net revenue: 17500 − 309000 = **−291500.00 (−94.34%)**. A baseline of −100 and current −50 means a +50% improvement.

> **previous = 0 → null percentage.** If either period has no AOV, its comparison is also null. Totals are compared as given: different period lengths are not normalised per day.

26 Group contributions

Contributions explain the arithmetic of change, not customer behaviour.

Allowed dimensions: **category** is the product category; **region** is the customer's region. Groups come from both periods, including refund-only groups. Sort order is delta ascending, then name.

> **Contribution, % = group net delta / total net delta × 100.** Signs are preserved. Zero total change gives null for every share.

October vs September: sample fixture

Category	Delta, RUB	Share
Pumps	−300000.00	102.92%
Sensors	−9500.00	3.26%
Valves	+18000.00	−6.17%
Total	−291500.00	100.01%

Pumps account for more than 100% of the decline: valves offset part of it and have a negative share of the overall decrease. Sensors remain in October through a refund only. Rounded shares sum to 100.01% rather than exactly 100%; this is expected.

```
uv run pytest tests/test_analytics_models.py \
  tests/test_analytics_cli.py
uv run pytest tests/integration/test_analytics.py
```

Integration tests use TEST_DATABASE_URL ending in _test only. They cover manual totals, double counting, rounding, Moscow boundaries, empty months, comparisons and contributions. Actual outcomes are in section 44 and docs/VERIFICATION.md.

> **Following stages:** anomaly detection is Stage 05; the metrics REST API is Stage 06; the LLM agent is Stage 07. A group's revenue change alone does not prove why sales fell.

Full seed 42 dataset: September vs August

The same CLI on the 20,000 Stage 03 orders loaded into a separate empty *_test database. Actual output on 21 October 2026:

```
uv run python -m app.analytics \
  --start 2026-09-01 --end 2026-10-01 \
  --compare-start 2026-08-01 --compare-end 2026-09-01 \
  --dimension category
```

Metric	August	September	Change
Paid sales	633 733 255.24	468 349 280.42	−165 383 974.82 (−26.10%)
Refunds	1 557 126.03	1 670 523.07	+113 397.04 (+7.28%)
Net revenue	632 176 129.21	466 678 757.35	−165 497 371.86 (−26.18%)
Paid orders	1 985	1 931	−54 (−2.72%)
Average order value	319 261.09	242 542.35	−76 718.74 (−24.03%)

Category	Net delta	Share
pumps	−138 597 069.79	83.75%
compressors	−26 317 759.86	15.90%
sensors	−294 972.94	0.18%
valves	−287 569.27	0.17%

This is scenario A01. Order count barely moved (−2.72%) while average order value fell 24%: fewer pumps are bought. Pumps account for 83.75% of the revenue decline. The analytics found it without reading ground_truth.json. By region the decline is spread evenly, from 7.72% (Ural) to 18.56% (Central).

The January–September monthly series also shows scenario **A03**: July refunds of 132,586,941.37 against 0.8–1.7 million in other months. This is measurement, not explanation: why pumps fell (stock, price, demand) does not follow from these numbers. Automatic detection of deviations is Stage 05.

27 Statistical candidates

An unusual observation is a reason to inspect data and hypotheses.

Stage 05 reads the current database and returns **candidates**. Rules do not know scenarios A01–A05. SQL/Decimal computes money; no LLM is involved. The settings below are explicit demo defaults.

Method	What is checked
Median and MAD	Monthly quantity and refund ratio by category; discount rate by region/category. At least 6 other months of the same series; $\|z\| \geq 6$.
MAD = 0	Difference strictly exceeds max(25% of |median|, floor). floor = 1 for quantity, 0.01 for rates.
IQR	Paid line quantity above Q3 + 3 × IQR within its category; at least 8 lines.
Zero inventory	A maximal run of ≥ 3 consecutive zero-stock days. A missing day breaks the run.

A month does not set its own baseline

The median is the middle value; MAD is the median absolute deviation from it. The scored month is excluded from its baseline (leave-one-out): $z = 0.6745 \times (x - \text{median}) / \text{MAD}$. Example: median = 100, MAD = 2, x = 60 → z = −13.49, a candidate at $|z| \geq 6$.

> **This is retrospective.** Other months may follow the month being scored. Monthly detectors use only complete months: clipped boundary months are skipped. Short histories may produce no score. This is not forecasting.

28 Isolation Forest

The model finds feature combinations; financial amounts remain exact.

scikit-learn **Isolation Forest** complements simple rules. One row is a complete month, region and category. At least 16 usable rows are required; entirely empty groups are not replaced with zeros.

Feature	Meaning
Quantity / median	Relative to its own series.
Paid sales / median	A ratio of exact amounts.
Discount rate	Weighted by gross value.
Refunds / sales	Undefined without sales.

SQL aggregates sales and refunds independently. Monetary amounts remain NUMERIC/Decimal. Only the calculated **dimensionless ratios** become floats for the model.

Reproducible run

100 trees; random_state = 42; n_jobs = 1; contamination = auto. Candidates have decision_function < 0. The score is neither a probability nor a monetary loss. Fitting and scoring use the supplied period; there is no separate unseen holdout.

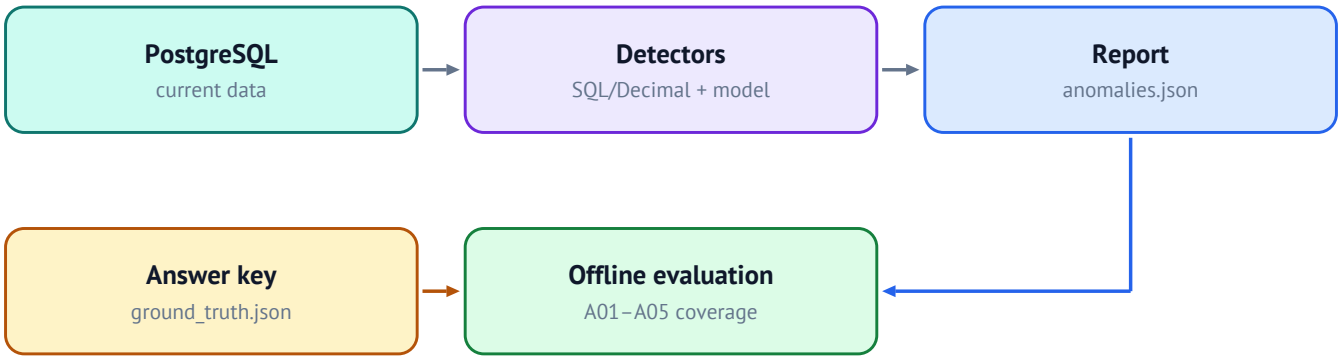
```
uv run python -m app.analytics.anomalies \
  --start 2026-01-01 --end 2026-10-01 \
  --output /tmp/anomalies.json
```

Reads use REPEATABLE READ READ ONLY via DATABASE_URL. Without --output, JSON goes to the terminal; existing files are never overwritten. The report includes period, seed, observation counts, settings and findings with method and evidence.

> **A candidate is not a cause.** Zero inventory and lower sales close in time do not establish a causal link. Additional candidates require inspection of the underlying rows.

29 Offline scenario evaluation

Only the separate offline evaluation command reads the answer key.




```
uv run python -m app.analytics.evaluate \
  --report /tmp/anomalies.json \
  --ground-truth data/demo/ground_truth.json
```

The report and answer key must describe the same dataset. Matching uses period, signal kind and dimensions; monthly scenarios also require the correct direction of change. Full local run on 21 Oct 2026: **33 candidates, 5/5 scenarios**.

Scenario	Event to check
A01	Pumps: quantity, September.
A02	Valves, South: discounts, August.
A03	Compressors: refunds, July.
A04	One pump: zero stock, 10–19 September.
A05	Sensors: a 500-unit line, June.

7 monthly-statistic candidates + 1 IQR + 1 stock rule + 24 Isolation Forest. Statistics and rules covered the scenarios; this does not evaluate the model alone. Two seed 42 runs produced identical JSON.

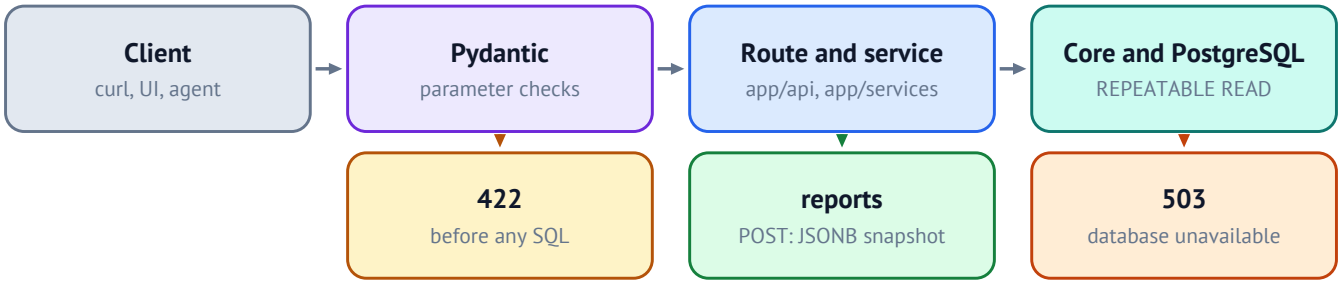
> **Coverage ≠ precision.** The answer key contains five injected positive scenarios, without labels for every ordinary observation. Even 5/5 does not mean 100% precision or quality on future data. An additional candidate is not automatically a false positive.

Checks cover positive and ordinary cases, zero MAD, short history, IQR, inventory gaps, model reproducibility and exact SQL features. Integration tests use only TEST_DATABASE_URL ending in _test. Results: section 44 and docs/VERIFICATION.md. Next is the REST API in Stage 06.

30 REST API and parameters

A thin HTTP layer over the existing analytics: the core computes numbers.

Stage 06 exposes the Stage 04–05 results over HTTP: the UI (08) and the agent (07) share one JSON format. A route in **app/api/** only accepts the request; a service in **app/services/** arranges the answer from **app/analytics/** functions. Neither computes money itself.



Method and path	Returns
`GET /api/metrics/revenue`	Sales, refunds, net revenue, months, comparison.
`GET /api/metrics/orders`	Paid orders and average order value.
`GET /api/analytics/categories`	Revenue change and each category's contribution.
`GET /api/analytics/regions`	The same by customer region.
`GET /api/analytics/anomalies`	Stage 05 candidates.
`POST /api/reports`	Compute and save a report.
`GET /api/reports/{id}`	A saved report as stored.

Parameters are a Pydantic model

```
@router.get("/revenue", responses=DATABASE_RESPONSES)
def revenue(
    session: ReadSession, query: Annotated[OptionalComparisonQuery, Query()]
) -> RevenueResponse:
    return analytics.revenue(session, query)
```

Query() maps model fields to query string parameters. A period is half-open in Moscow time: start=2026-09-01&end=2026-10-01 is September. Rules: start < end, at most 120 calendar months, compare_start and compare_end together or not at all, seed 0–4294967295. An unknown parameter (extra="forbid") is also a 422, so a typo never becomes a silently ignored filter.

> **Money is a JSON string.** Pydantic writes Decimal as "466678757.35", so a client float never loses kopecks. An undefined value (AOV without orders, a percentage of a zero baseline) is null. Every response states "currency": "RUB" and "timezone": "Europe/Moscow".

31 Sessions and errors

One data snapshot per request; database errors never reach the client.

A route declares what it needs and FastAPI calls the dependency through Depends. **ReadSession** for every GET is a REPEATABLE READ READ ONLY transaction: all queries of one response see the same data and PostgreSQL itself rejects writes. **WriteSession** stores reports and agent runs; the code commits explicitly and an error rolls back. Tests replace both through app.dependency_overrides with a *_test database session.

Finding: dependencies run before validation fails

The first version opened an ordinary Session. Tests showed that FastAPI calls dependencies even when parameters are invalid. Without DATABASE_URL a request with end ⩽ start got 503 instead of 422, and with a database it opened a needless connection. The fix is **LazySession**: the engine is chosen at the first SQL statement and cached. A stable bind makes later statements reuse the same connection.

```
class LazySession(Session):
    # __init__: self._resolved_engine = None
    def get_bind(self, mapper=None, **kw):
        if self._resolved_engine is None:
            engine = get_engine()
            self._resolved_engine = (
                engine.execution_options(
                    isolation_level="REPEATABLE READ",
                    postgresql_readonly=True,
                ) if self.read_only else engine
            )
        return self._resolved_engine
```

Why def and not async def. Session and psycopg are synchronous; FastAPI runs a plain function in a thread pool, so waiting for the database does not block other requests. async def with a synchronous session would block the whole server's event loop. Stage 07 applies the same approach to the synchronous model SDK.

Code	When
200 / 201	A read; a report saved, its address in the Location header.
404	Unknown path or report UUID.
422	Invalid parameters or body; no SQL runs.
503	No DATABASE_URL, database unreachable or connection lost.

One 503 text. The client gets {"detail": "Database is not ready"} for any cause; the database address and driver message stay in the server log. A test replaces get_engine with a function that fails and checks 422 for 17 invalid requests, so none of them reached the database.

32 Saved reports

A report is an immutable snapshot: parameters, result and code version.

Migration **0002** adds the reports table. It has no foreign keys to sales rows: a report stores a finished API answer, not links to rows that may change.

Column	Meaning
`id`	UUID, set on save.
`kind`	comparison or anomalies; a database CHECK.
`parameters`	The request body as JSONB.
`result`	The answer in API format as JSONB.
`app_version, created_at`	Code version and time; an index on created_at.

```
curl -i -X POST http://127.0.0.1:8000/api/reports \
  -H 'Content-Type: application/json' \
  -d '{"kind": "comparison", "start": "2026-09-01", "end": "2026-10-01",
       "compare_start": "2026-08-01", "compare_end": "2026-09-01"}'
```

The **kind** field is a discriminator: Pydantic picks the body model by it (a comparison with dimension or anomalies with seed). The answer is 201 with location: /api/reports/<uuid>. GET on that address returns the same JSON; the GET /api/reports list is newest first, limit 1–100, without result bodies.

A snapshot, not a link

A test saves the October vs September comparison on the sample: −291,500.00. Then it adds a 50,000.00 October sale. The live route shows −241,500.00 while the saved report still shows −291,500.00.

The full dataset through the API

Demo request, seed 42	Result	Time, approx.
Revenue, Sep vs Aug	−165,497,371.86 (−26.18%)	30 ms
Orders and AOV	1,931; 242,542.35 (−24.03%)	16 ms
Category contributions	pumps 83.75%	14 ms
Anomalies, Jan–Sep	33 candidates	177 ms
`POST /api/reports`	201, GET returned the same body	22 ms

There is no authentication yet. Run the server on 127.0.0.1 only. Reports cannot be deleted through the API and the list has no total count. Network access comes after Stage 09.

Checks: API numbers equal the core functions, regions reconcile with the total, 422 without SQL, 503 without a database address, writes in ReadSession are rejected, migration 0002 rolls back alone. Results: section 44 and docs/VERIFICATION.md. Next is the AI agent in Stage 07.

33 Agent tools

The model picks a function from a list; it never writes SQL.

Stage 07 answers plain-language questions such as “Why did revenue fall in September?”. The language model understands the question, picks tools and phrases the conclusion. Code still computes every number: the tools use REST API functions and exact SQL queries.

Tool	Returns
`get_data_coverage`	Dates of sales, refunds and stock; category and region names.
`get_revenue`	Period totals and months.
`compare_periods`	All metrics of two periods and changes.
`get_sales_by_category`	Revenue change by category and shares.
`get_sales_by_region`	The same by region.
`get_top_products`	Top-selling products.
`get_refund_statistics`	Refunds per category and their rate to sales.
`detect_anomalies`	Stage 05 candidates: rules, then Isolation Forest.
`get_inventory_history`	Zero-stock days per product.

Strict arguments

```
class ListArguments(PeriodArguments):
    category: str | None = Field(max_length=100)
    limit: int = Field(ge=1, le=20)
# model_config: extra="forbid", strict=True
```

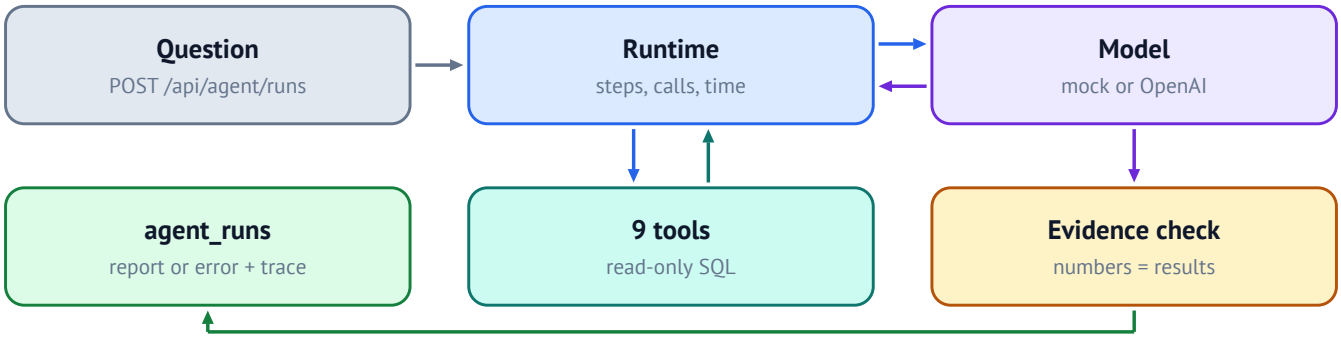
The model becomes a JSON Schema with **strict: true**: the model must send arguments exactly by the schema. The server validates them again anyway, before any SQL. An extra field, a number as text, a repeated key, a reversed period or more than 24 months returns an error the model can correct. A test covers such cases and forbids even opening a database session.

```
{"ok": false, "error": {"code": "invalid_arguments",
 "message": "period: start must be earlier than the exclusive end"}}
```

Why no free-form SQL. Heavy queries, extra tables and the double counting of Stage 04. The tool name selects a dictionary entry, arguments are data, and the query is pre-written and runs in a REPEATABLE READ READ ONLY session. A result over 16,000 characters becomes an error: it would not fit the context.

34 Tool-calling loop

The server owns the loop; the model only proposes the next step.



On every step the runtime sends the whole history: the question, earlier calls and their results. The model answers with tool calls, several at once, or with a report. Each result goes back with its **call_id**. The second step depends on the first: comparison, categories, regions and refunds first, then anomalies and stock of the main category.

Limit	Default	What it bounds
`AGENT_MAX_STEPS`	8	Model answers
`AGENT_MAX_TOOL_CALLS`	16	Calls; an over-limit batch is rejected whole
`AGENT_MAX_CORRECTIONS`	2	Report retries after the check
`AGENT_TIMEOUT_SECONDS`	120	Overall deadline checked between operations
context	120,000 chars	Whole history; more is 413

store=False. The response is not saved for a later conversation turn; the whole history is sent again, including the model's encrypted reasoning. The server sets finite step and call limits. The overall deadline is checked between operations; active synchronous SQL is not forcibly interrupted. Tests check limits with a scripted provider, without a network or randomness.

35 Report and evidence

Recognised numeric literals are matched to evidence.

AnalysisReport field	Content
`language`	ru or en: the question's language.
`summary`	Short answer in the question's language.
`facts`	Statements with evidence references.
`hypotheses`	Possible explanations, not proven causes.
`recommendations`	What to check or do next.
`limitations`	What the analysis does not cover.


```
{"call_id": "mock-category", "field": "contributions.0.delta",
 "value": "-138597069.79"}
```


Before a report is accepted, **check_report** applies two rules. 1) Evidence is real: the call happened in this run, succeeded, the path leads to one value, and it matches exactly; every fact has at least one evidence item. 2) Recognised numbers match: in facts and hypotheses by their own evidence, in the summary, recommendations and limitations by any verified evidence. The sign may be dropped and digits grouped. ISO dates, calendar-context years, codes such as A01 and digits in cited names are excluded. An amount of 2000 RUB still needs evidence.

Report text	Check
“fell by 291,500.00 (−94.34%)”, both in evidence	accepted
“fell by 300000.00”, evidence −291500.00	`number '300000.00' is not in its evidence`
evidence −291000.00, result −291500.00	`is '-291500.00', not '-291000.00'`
summary: “that is 95%”	`number '95' is not in any evidence`

Mismatch means a correction. The model gets the list of problems and sends the report again, at most twice; then a 502. The check does not prove metric identity, sign or causation; number words and other notations may not be recognised. Causes are written as hypotheses. Found during development: “Demo pumps 01” holds name digits, not a figure; names from the evidence are removed before numbers are searched.

36 Providers and history

Mock for a keyless demo, OpenAI only when configured.

Provider	How it works
mock	Default. Four steps over the real tools; reads the month, year and language of the question. The report passes the same check and is marked as a demo.
openai	Responses API: strict tools, parallel_tool_calls, a strict json_schema report, store=False, a fixed URL, no retries and no default model.

```
curl -X POST http://127.0.0.1:8000/api/agent/runs \
  -H 'Content-Type: application/json' \
  -d '{"question": "Почему в сентябре снизилась выручка?"}'
```

Investigation history

Successes and caught LLM failures are saved in **agent_runs** (migration 0003) when storage succeeds: question, provider, model, report or error, processed calls, tokens and time. A 502/503/504/413 then includes a run_id. Pre-run settings failures and an unavailable database have no stored run. GET /api/agent/runs lists runs; GET /api/agent/runs/{id} returns the full saved run. JSONB(none_as_null=True) writes None as SQL NULL; the CHECK is tested.

Full dataset, mock provider

Question	Report highlights	Time
Why did revenue fall in September?	−165,497,371.86 (−26.18%); pumps 83.75%; Demo pumps 01 — 10 zero-stock days	311 ms
Why did revenue fall in July 2026?	−80,258,104.27 (−13.60%); compressors 99.81%; refunds 30.28%	251 ms

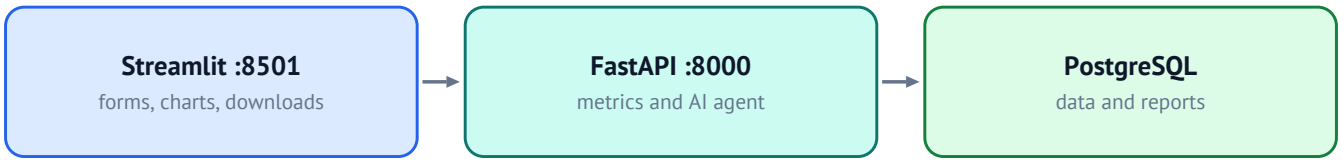
A real OpenAI model was not tested: there is no key. The request format, response parsing and SDK errors were tested with a fake SDK. The route stays def: the SDK and the database are synchronous, FastAPI runs the request in its thread pool. No authentication yet: 127.0.0.1 only.

Mock examples show signs of A01, A03 and A04 without reading ground_truth.json; this is not an accuracy evaluation. Results: section 44 and docs/VERIFICATION.md. Next is the Streamlit UI in Stage 08.

37 Streamlit interface

A browser interface over the existing REST API, using the same analytics core.

Stage 08 makes metrics, agent questions and saved reports available in a browser. **Streamlit** builds the page from Python code in **ui/main.py**. User actions call the HTTP client in **ui/client.py**, which talks to FastAPI. PostgreSQL and the model SDK run on the API server. Importing shared response schemas also imports model metadata, but the UI creates no database session and opens no database connection.



HTTP + JSON · the UI never opens a database connection

Run in two terminals

```
uv run uvicorn app.main:app --reload
# In another terminal, from the same project root:
uv run python -m streamlit run ui/main.py --server.address 127.0.0.1
```

First prepare .env, start PostgreSQL, apply migrations and load the demo as explained in earlier stages. Confirm that **http://127.0.0.1:8000/ready** returns 200. After starting Streamlit, open **http://127.0.0.1:8501**. It is a separate process: stopping the API leaves the page open, but new requests show a connection error.

Setting	Purpose
`UI_API_URL`	Local FastAPI address; http://127.0.0.1:8000 by default, without credentials or a path.
`UI_REQUEST_TIMEOUT_SECONDS`	Response timeout: 150 s by default, range 1–600 s; does not cancel an active agent run.
`LLM_PROVIDER=mock`	An API setting: the teaching scenario needs no key or OpenAI request.

The model key belongs on the API side. The interface does not ask for an OpenAI API key. It displays the server's result. Mock follows the fixed Stage 07 scenario rather than understanding an arbitrary conversation. Authentication and release belong to later stages; the current launch listens on loopback.

38 Metrics and charts

Exact labels and tables; approximate coordinates only for drawing.

Select **Dashboard** in the sidebar, then a half-open period: the start is included and the end excluded, in Europe/Moscow time. September is **2026-09-01 → 2026-10-01**. Comparisons, categories, regions and anomaly candidates come from the Stage 06 routes. A form button sends the chosen parameters. Monthly charts show revenue and orders, while category and region bars show signed contributions to the change. Anomalies use a separate form.

Layer	Money representation
PostgreSQL / Python	NUMERIC and Decimal: the server computes sums, shares and changes.
JSON / UI	A decimal string; the label keeps kopecks and null means unavailable.
Plotly	A float sets a point or bar height; exact text is supplied separately in customdata.

Why a chart has two representations

```
money = Decimal("466678757.35")
y_coordinate = float(money)
exact_label = format(money, ",.2f")
# Plotly: y=[y_coordinate], customdata=[exact_label]
# Hover text uses %{customdata}, not a recalculated total.
```

Plotly needs a numeric array to position marks on an axis. The conversion to float serves only that drawing. Totals are never recalculated from coordinates; cards, tables, hover labels and downloaded JSON retain the API values. If a percentage is undefined because the baseline is zero, the UI displays an unavailable value instead of replacing it with zero.

> **Visual check.** Compare one card, table and chart hover value with GET /api/metrics/revenue in /docs. Then choose a period with no sales: the UI should show an empty result without invented figures. An anomaly candidate remains a signal to investigate, rather than a proven cause of a decline.

39 Questions and reports in the UI

One question creates one agent run; saved history lives in the API.

Select **AI analyst**. Enter, for example, “Why did revenue fall in September 2026?” and submit the question. The UI sends **POST /api/agent/runs** and displays the finished report: a summary, facts with evidence, hypotheses, recommendations and limitations. Each is a separate run; a chat-like display does not give the model memory of past questions. Saved history lets you open an earlier run and see its result. Widgets rerun the page's Python code; session_state keeps the submitted query and response, so another POST requires submitting the form.

Action	What happens
Ask a question	POST creates a run; the model and tools execute on the API server.
Open history	GET reads the list and saved run by UUID without another model call.
Save analytics	In Dashboard: POST /api/reports recomputes and saves a period snapshot.
Download JSON	A copy of the API response with exact decimal strings and metadata.

How to interpret errors

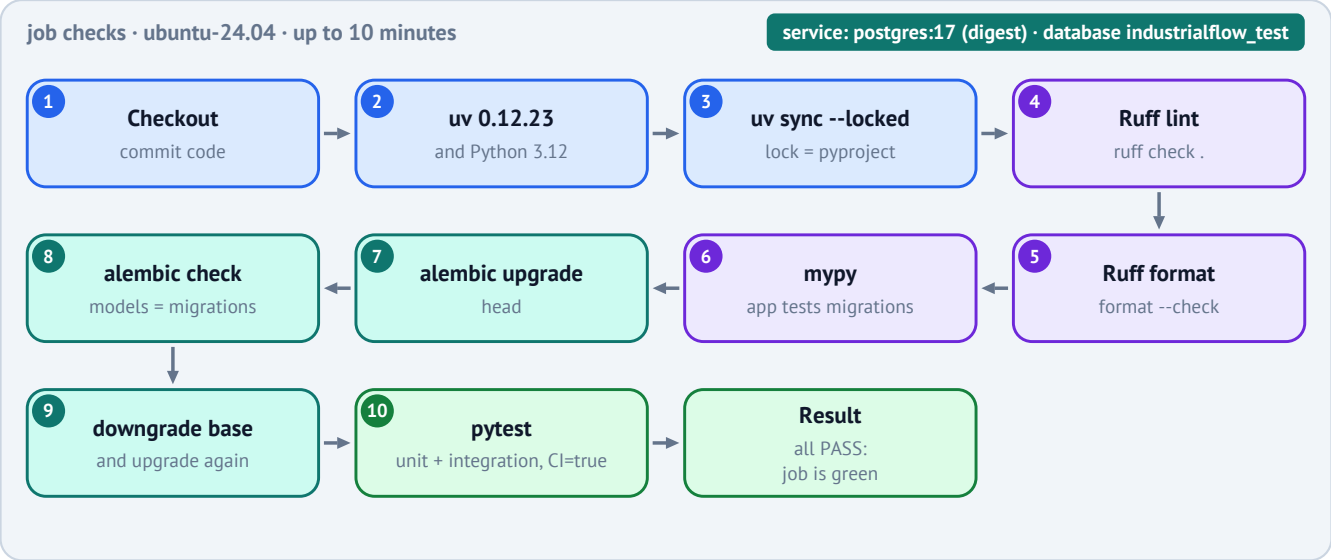
Dashboard metrics arrive in separate GET requests, each with its own database snapshot. Saving a report runs a new calculation and displays the returned snapshot: if new sales arrived, it may differ from the earlier Dashboard.
No connection: check the API address and Uvicorn process. 422: correct the parameters or question; 404: the saved UUID does not exist; 503: check /ready, settings and the database. A provider failure may return 502 and a deadline failure 504. A UI HTTP timeout does not prove the server stopped working: refresh run history before retrying a POST. Stored errors are available only in the cases described in Stage 07.

> **Try it yourself.** Open September, ask a question, download JSON and find the same UUID in history. Save an ordinary analytics report and reopen it: a read should return the saved snapshot. Stop the API, perform another action and check the clear error message. The checks actually performed are listed in section 44.

40 CI on GitHub Actions

The same checks on a clean Linux machine with its own PostgreSQL.

Runs on: push to main · pull request to main · manual: workflow_dispatch



Setting	Why
`services: postgres`	A dedicated industrialflow_test database for the run; image pinned by digest as in compose.yaml.
`DATABASE_URL, TEST_DATABASE_URL`	Alembic and tests use the same *_test database.
`CI=true`	Integration tests without an address fail instead of being skipped.
`permissions: contents: read`	The job may only read the code.
`checkout@3d3c42e… # v7.0.1`	Actions are pinned by full SHA: a tag can move, a SHA cannot.
`concurrency`	A new push to the same PR cancels the outdated run.

Remote CI status: not confirmed. Stage 08 passed locally; its remote run is not confirmed. Local success does not replace GitHub Actions.

41 Command cheat sheet

Essential commands for Stages 01–08.

Task	Command
Create the environment from the lock	`uv sync --locked`
Start PostgreSQL	`docker compose up --detach --wait db`
Apply migrations	`uv run alembic upgrade head`
Generate demo	`uv run python -m app.data.generate --output data/demo`
Load demo	`uv run python -m app.data.load --input data/demo`
Data quality	`uv run python -m app.data.quality`
September metrics	`uv run python -m app.analytics --start 2026-09-01 --end 2026-10-01`
Anomaly candidates	`uv run python -m app.analytics.anomalies --start 2026-01-01 --end 2026-10-01`
Start the server	`uv run uvicorn app.main:app --reload`
Check readiness	`curl -i http://127.0.0.1:8000/ready`
Start the interface	`uv run python -m streamlit run ui/main.py --server.address 127.0.0.1`
Revenue via the API	`curl '127.0.0.1:8000/api/metrics/revenue?start=2026-09-01&end=2026-10-01'`
Saved reports	`curl '127.0.0.1:8000/api/reports?limit=10'`
Agent history	`curl '127.0.0.1:8000/api/agent/runs?limit=10'`
Test database	`docker compose --profile test up --detach --wait db-test`
All tests	`TEST_DATABASE_URL=... uv run pytest`
Integration only	`uv run pytest -m integration`
Lint and format	`uv run ruff check . && uv run ruff format --check .`
Types	`uv run mypy app ui tests migrations`
Scenario evaluation	`uv run python -m app.analytics.evaluate --help`
Models = migrations?	`uv run alembic check`
Roll back one migration	`uv run alembic downgrade -1`
Schema version	`uv run alembic current`
Stop the database	`docker compose stop`
Check the lock	`uv lock --check`
Rebuild the PDF	`uv run --with 'reportlab==4.4.9' python scripts/build_guide.py`

42 Common errors

Symptom, cause and fix.

Symptom	Cause	Fix
`uv: command not found`	Terminal opened before installing uv	Open a new terminal tab
`No module named 'app'`	No pythonpath or not run from the root	Check pyproject.toml, cd to the root
`Address already in use`	Port is busy	`lsof -i :8000`, `--port 8001`
`405 Method Not Allowed`	Wrong HTTP method	Use GET for /health
`uv sync --locked` fails	pyproject changed, lock did not	`uv lock` and commit uv.lock
.env changes ignored	Settings are read at startup	Restart the server
`failed to connect to the docker API`	Docker Desktop is not running	Start Docker Desktop
`POSTGRES_PASSWORD is missing`	No .env	`cp .env.example .env`
`port is already allocated`	Port 5436 is busy	Change POSTGRES_PORT and DATABASE_URL
`DATABASE_URL не задан`	No .env or variable	Create .env
/ready → 503 with a live database	Migrations not applied	`uv run alembic upgrade head`
`New upgrade operations detected`	Model changed without a migration	A new --autogenerate migration
Integration tests skipped	No TEST_DATABASE_URL	Start db-test, set the variable
“only for a database ending with _test”	The address points to another database	Use a *_test database
`IntegrityError`	An integrity rule is violated	Read the constraint name in the error
The API answers 422	Extra parameter, end ⩽ start, > 120 months	Read detail[].loc and msg
The API answers 503 to everything	No database or DATABASE_URL	Start db, check /ready
Data is gone	`docker compose down -v`	Without -v the volume stays

43 Check yourself

Self-check questions and practice tasks.

1. How does pyproject.toml differ from uv.lock?
Answer: pyproject holds requirements for direct dependencies; uv.lock holds exact versions and hashes of all packages.

2. Why does Uvicorn find app while pytest without a setting does not?
Answer: Uvicorn adds the current folder to sys.path, pytest adds tests/. pythonpath = ["."] adds the root.

3. What happens if health() returns {"status": "fail"}?
Answer: mypy finds it before running; on a request FastAPI raises ResponseValidationError and the client gets 500.

4. Why is an order stored in two tables?
Answer: One order holds many products; normalisation stores order properties once and products line by line.

5. How does CASCADE differ from RESTRICT?
Answer: CASCADE deletes dependent rows with the parent; RESTRICT forbids deleting a referenced parent.

6. Why is money NUMERIC and Decimal?
Answer: float is binary and cannot store 0.1 exactly; NUMERIC and Decimal store decimals exactly.

7. Why AT TIME ZONE 'Europe/Moscow'?
Answer: Reporting months are Moscow time: a payment at 01:30 MSK on 1 October would fall into September in UTC.

8. Why is an applied migration never edited?
Answer: Different databases would share a version number; a change is a new migration.

9. Why does a JOIN with refunds inflate sales?
Answer: The line repeats once per refund; reduce the “many” table to one row per key first.

10. How does /ready differ from /health?
Answer: /health — the process is alive, no database needed; /ready — database reachable and current, otherwise 503.

11. Why does an invalid request get 422 even when the database is down?
Answer: Pydantic validates parameters before the route, and LazySession picks the engine only at the first SQL statement.

12. Why does a saved report not change after new sales?
Answer: The result is computed once and stored as JSONB; GET returns the snapshot, not a recalculation.

Practice tasks

Stage 01. A GET /version route with a VersionResponse model: name from get_settings().app_name, version from app.__version__; a test for 200 and the body.

Stage 02. A products.is_active column (boolean, NOT NULL, default true) via a new migration 0004 with server_default; alembic check is quiet, downgrade -1 works, a test exists.

Stage 06. A DELETE /api/reports/{report_id} route: 204 after deletion, 404 for an unknown UUID; a test that the next GET is 404 and GET routes still cannot write.

44 Verification results

Verified on 2026-10-21; working-tree base 2e34253 + worktree.

Environment: macOS, Apple Silicon, uv 0.12.23, Python 3.12.15, Docker 29.8.2, PostgreSQL 17. Tests: 484 passed (369 unit + 115 integration), 0 failed, 0 skipped.

Check	Command	Result	Status
Tests	`CI=true TEST_DATABASE_URL=... pytest`	484 passed: 369 unit + 115 integration	PASS
Lint / format	`ruff check; ruff format --check`	All checks passed; 148 files	PASS
Types	`mypy app ui tests migrations`	No issues in 98 files	PASS
Migrations	`alembic upgrade / check / downgrade`	Schema 0003; no new migrations	PASS
Dependencies	`uv lock --check --offline`	79 packages; Streamlit 1.65, Plotly 7.1	PASS
Offline UI	`pytest tests/test_ui_*.py`	109 tests; 16 use AppTest	PASS
Browser	`Streamlit + FastAPI, seed 42`	20,000 orders; 3 views; mock response	PASS
Reports	`POST reports; GET saved UUID`	2 kinds; reopened without recomputing	PASS
No API	`Stop Uvicorn; Load dashboard`	Safe error without traceback	PASS

Server responses

Request	Code	Body
`GET /health`	200 OK	`{"status":"ok"}`
`GET /ready`	200 OK	`{"status":"ready","revision":"0003"}`
`GET /api/metrics/revenue`	200 OK	`net_revenue: "466678757.35"; change: -26.18%`
`GET /api/analytics/categories`	200 OK	`pumps: -138597069.79; contribution: 83.75%`
`GET /api/analytics/anomalies`	200 OK	`finding_count: 33; seed: 42`
`POST /api/agent/runs`	201 Created	`mock; 4 steps; 7 calls; 248 ms`
`POST /api/reports`	201 Created	`Comparison and anomalies; then GET by UUID`
`UI: API stopped`	Connection error	`Cannot reach the API. Check that it is running.`
`POST /health`	405 Method Not Allowed	`{"detail":"Method Not Allowed"}, allow: GET`
`GET /nope`	404 Not Found	`{"detail":"Not Found"}`

GitHub Actions: not confirmed. Stage 08 passed locally; its remote run is not confirmed. Local success does not replace GitHub Actions.

What is next

- Stage 09: agent evaluations and security.
- A real OpenAI model was not tested: no key. Confirm remote Stage 08 CI.
- The UI does not send conversation memory. No authentication yet: loopback only; an HTTP timeout does not cancel API work.